

Análisis y abuso de reglas sudo

Objetivos de aprendizaje

Leer usuario destino, comando, argumentos, tags y entorno de una regla sudo; derivar la capacidad real y validar dependencias de versión sin asumir shell root.

Prerrequisitos

UID, argv, entorno, rutas, permisos y documentación de sudoers.

Fundamentos técnicos

Una regla expresa quién, dónde, como quién y qué puede ejecutar. NOPASSWD elimina una autenticación, no las restricciones de comando. Argumentos fijos, NOEXEC, SETENV, secure_path, versión y capacidades del binario cambian el resultado.

Modelo mental

regla -> runas + ruta + argumentos + tags + entorno
-> comportamiento documentado del binario -> capacidad mínima

Superficie de ataque

sudo -l, includes de sudoers, reglas de grupo, sudoedit y entornos preservados. Se analiza la salida completa.

Cómo identificarla

- La invocación exacta coincide con la regla.
- El binario ofrece una función de lectura, escritura o ejecución alcanzable con argumentos permitidos.
- La versión instalada conserva esa función.
- Un marcador confirma la identidad destino.

Preguntas que debo hacerme

1. ¿Runas es root u otro usuario?
2. ¿Argumentos son libres, fijos o comodines?
3. ¿Qué tags y Defaults aplican?
4. ¿Qué entorno llega al proceso?
5. ¿Qué capacidad mínima aporta el binario?

Prueba mínima

Ejecutar la invocación permitida de forma no destructiva y confirmar EUID o acceso al recurso previsto. Consultar manual local y versión antes de recetas externas.

Construcción progresiva del payload

1. Copiar regla exacta.
2. Separar host, runas, tags, ruta y argumentos.
3. Reproducir operación legítima.
4. Identificar función secundaria alcanzable.
5. Usar marcador id o fichero temporal.
6. Comparar con argumento que no coincide.

Anatomía de los payloads

- Contexto de entrada: (backup) NOPASSWD: /usr/bin/cat /srv/app/report.txt.
- Sintaxis original: ruta y argumento exactos.
- Entrada controlada: ninguna; solo ejecución autorizada.
- Transformaciones conocidas: sudo compara ruta, RunAs y argumentos antes de ejecutar.
- Parser final: sudoers y luego cat.
- Sink: lectura como backup.
- Primitiva: leer ese fichero con identidad destino.
- Payload mínimo: invocación exacta.
- Significado de cada componente: sudo -u backup selecciona RunAs; ruta y operando reproducen la regla exacta.
- Resultado esperado: lectura del reporte como backup y rechazo de cualquier operando distinto.
- Control negativo: sustituir ruta por otro fichero.
- Restricción observada: argumento fijo.
- Por qué falla la variante básica: pedir /etc/shadow no coincide.
- Hipótesis de adaptación: estudiar si el fichero permitido es enlace/escribible; no inventar argumento.
- Payload adaptado: solo si otra precondition real lo permite.
- Por qué debería funcionar: la adaptación debe actuar sobre un componente controlable que la regla realmente consuma.
- Evidencia: lectura del reporte y rechazo del control.
- Cuándo no funcionaría: regla distinta, contraseña requerida o política adicional.

Variaciones según el contexto

NOEXEC intenta impedir que ciertos ejecutables lancen otros procesos, con dependencias de plataforma/compilación. SETENV permite mayor control ambiental; secure_path sustituye PATH salvo excepciones documentadas. Debe comprobarse la versión local.

Filtros y bypasses

Una regla con argumentos exactos no se “salta” añadiendo texto al azar. Los comodines de sudoers y del programa son capas distintas. Analiza qué cadena compara sudo y qué parser consume después.

Evidencias de confirmación

Capacidad ejecutada como runas y atribuida a una función permitida. Ver NOPASSWD no confirma escape.

Escalado de impacto

De lectura a escritura o ejecución solo si la regla y el binario lo permiten. GTFOBins sirve como índice, no como sustituto del manual y la versión.

Errores frecuentes

- Leer solo NOPASSWD.
- Ignorar runas no root.
- Cambiar ruta/argumentos y esperar coincidencia.
- Suponer que variables peligrosas sobreviven.
- Ignorar NOEXEC y versión.

Diagnóstico de payloads fallidos

Síntoma	Hipótesis	Prueba
"not allowed to execute"	ruta/argumentos no coinciden	copiar regla y usar <code>sudo -l</code>
función de escape no abre proceso	NOEXEC, versión o build	tags, <code>sudo -V</code> , manual local
PATH manipulado no cambia binario	<code>secure_path</code> o ruta absoluta	<code>sudo -V</code> , entorno dentro del comando
variable desaparece	<code>env_reset/linker</code>	<code>sudo -V</code> y variable inocua
EUID es otro usuario no root	runas específico	<code>id</code> como ese usuario

Mitigaciones

Comandos mínimos con rutas y argumentos exactos, evitar binarios multipropósito, NOEXEC/intercept como defensa adicional, entorno saneado, `secure_path`, logging y revisión periódica.

Relación con pentesting y certificaciones

Se evalúa lectura precisa de la política y capacidad, no buscar cada nombre en GTFOBins.

Caso guiado

Caso A — Básico: lector como usuario backup

La regla permite cat de un reporte como backup. Experimento: invocación exacta y otro fichero como control. Resultado: solo el reporte. Conclusión: capacidad limitada de lectura, no root.

Caso de adaptación

Caso B — Editor con NOEXEC

Una receta conocida intenta lanzar un proceso y falla. Hipótesis: NOEXEC, versión o comando sin función. Experimento: revisar tag, build y una operación interna no ejecutora. Resultado: NOEXEC activo. Conclusión: la receta básica no aplica; se evalúan solo capacidades internas del editor.

Caso C — Transferencia: variable preservada

Una regla permite un script de diagnóstico y conserva una variable de configuración. El alumno demuestra con valor marcador que el script lee esa ruta como usuario destino; después comprueba permisos del archivo. La primitiva es controlar configuración consumida, no "sudo shell".

Caso D — Falso positivo: regla sin contraseña

(root) NOPASSWD: /usr/bin/systemctl status app.service muestra estado, pero argumentos adicionales no coinciden y el pager está deshabilitado. Conclusión: no hay escape demostrado.

Ejercicios

1. Descompón una regla con runas, tags y argumentos.
2. Diseña un control para `secure_path`.
3. Explica diferencia entre `SETENV` y `env_keep`.
4. Evalúa una receta dependiente de versión.

Resumen

Sudo concede una invocación descrita por política. El análisis parte de esa gramática y deriva una capacidad exacta.

Chuleta operativa

1. Runas.
2. Ruta.
3. Argumentos.
4. Tags.
5. Defaults/entorno.
6. Versión y función.
7. Marcador/control.

Referencias

- [sudoers\(5\)](#)
- [Concepto interno](#) (sudo-abuse)

Navegación

Anterior: [Enumeración](#). Siguiendo: [SUID](#). Índice: [curso](#).